GUÍA DE ESTUDIO · RETO TÉCNICO FULL STACK ENGINEER · IA AGÉNTICA

Storefront Guardian

Qué es, para qué sirve y cómo funciona por dentro, explicado parte por parte para alguien que recién empieza a programar.

Autor del proyecto: Fabrizio Cuassolo · Repositorio: github.com/fabriziocuassolo/Agente-Reviewer
Escrito para estudiar antes de la presentación en vivo de 30 minutos.

Cómo usar esta guía

Está pensada para leerse en orden. Los primeros capítulos no tienen código: explican la idea con ejemplos de la vida diaria. Recién después se entra al código, archivo por archivo, siempre con la misma ficha: **qué hace**, **una comparación simple** y **un pedacito de código explicado**.

Al final hay un capítulo para la entrevista: qué hiciste vos y qué hizo la IA, las preguntas más probables con respuestas cortas y un guion para la demo.

Cómo leer las cajas de colores

Azul idea clave · **Verde** ejemplo concreto · **Amarillo** ojo, detalle importante · **Violeta** frase lista para decir en la entrevista

1	El problema y la solución en una página	Sin código
2	Mini diccionario	Las 20 palabras técnicas que vas a escuchar
3	Qué hace Guardian: los comandos	Con ejemplos de uso
4	Casos de uso	Quién lo usa, cuándo y para qué
5	El viaje de un cambio	El "pipeline" en 5 pasos
6	Cómo está organizado el proyecto	El mapa de carpetas
7	La tienda de ejemplo	El "paciente" que revisa Guardian
8	El agente, archivo por archivo	El corazón del proyecto
9	Los 4 escenarios de demo	Qué bug tiene cada uno y quién lo encuentra
10	Demo, replay, hook y GitHub Action	Cómo se prueba y se integra
11	Por qué este stack y no otro	n8n, Google ADK, Power Apps…
12	Costo, tiempo, privacidad y operabilidad	Las 4 condiciones del reto
13	Qué hiciste vos y qué hizo la IA	Para responder con honestidad
14	Límites y qué harías con más tiempo	Incluye la versión "agéntica"
15	Preparación para la entrevista	Preguntas, respuestas y guion de demo
16	Machete de comandos	Una página para tener a mano

1. El problema y la solución en una página

El problema que plantea la empresa

Un equipo mantiene varias tiendas online. Cada vez que un programador cambia algo, otro compañero tiene que **revisar** ese cambio antes de que llegue a la tienda real. Eso tiene tres problemas:

- **Es lento:** el revisor no siempre está disponible.
- **Es inconsistente:** cada revisor mira cosas distintas.
- **Los errores se ven tarde:** un descuento mal calculado puede llegar a producción y costar plata real.

La solución: Guardian

Guardian es un **revisor automático**. El programador escribe `guardian review` en la terminal y, en segundos, recibe una lista de problemas con la explicación de cada uno y cómo arreglarlo. Al final da un veredicto:

Decisión	Qué significa	Ejemplo
APROBADO	El cambio puede avanzar.	Agregaste un filtro bien hecho y con tests.
ADVERTENCIA	Puede avanzar, pero hay cosas para mirar.	Falta un test para una función nueva.
BLOQUEADO	No debería avanzar hasta corregirlo.	Escribiste una contraseña dentro del código.

La comparación que conviene recordar: una fábrica con control de calidad

Imaginá una fábrica de tortas. Antes de que una torta salga a la venta pasa por tres controles:

1. **Las máquinas de medir** (balanza, termómetro): siempre dan el mismo resultado. En Guardian son los **chequeos determinísticos**: detector de contraseñas, revisión de estilo, tipos y tests.
2. **El catador experto**: prueba la torta y opina con criterio. En Guardian es la **inteligencia artificial** (el "LLM").
3. **El jefe de calidad con un reglamento escrito**: mira los informes y decide si la torta sale o no. En Guardian es el **gate de decisión**, que sigue reglas fijas.

El detalle importante: **el catador opina, pero no decide**. Decide el reglamento. Así cada decisión se puede explicar y repetir.

Qué capacidades del reto demuestra

El reto pedía al menos 2 de 4 capacidades. Guardian demuestra 3:

Capacidad del reto	¿Está?	Dónde
1. Revisar cambios y dar feedback útil	**Sí · núcleo**	`guardian review`
4. Decidir si un cambio está listo	**Sí · núcleo**	El gate: APROBADO / ADVERTENCIA / BLOQUEADO
3. Responder preguntas sobre el código	**Sí · versión simple**	`guardian ask`
2. Detectar, corregir *y validar* la corrección	**Parcial**	Propone la corrección, pero no la prueba. No hay que presentarla como cumplida.

2. Mini diccionario

Estas son las palabras que vas a usar y escuchar. No hace falta memorizarlas: volvé a esta página cuando aparezcan.

Palabra	Qué es, en simple
Repositorio (repo)	La carpeta del proyecto con todo su historial de cambios.
Git	El programa que guarda ese historial. Es como el "control de cambios" de Word, pero para código.
Commit	Una "foto" guardada del proyecto con un mensaje ("agrego filtro por categoría").
Push	Subir tus commits al servidor compartido (GitHub) para que los vea el equipo.
Diff	La lista de líneas que cambiaron: las agregadas con `+` y las borradas con `-`.
Rama (branch)	Una copia paralela del proyecto para trabajar sin tocar la versión principal (`main`).
Pull Request (PR)	El pedido de "sumá mis cambios a la versión principal". Ahí se hacen las revisiones.
Hook	Un script que git ejecuta solo en cierto momento. El *pre-push* corre justo antes de subir cambios.
CI / GitHub Action	Un robot de GitHub que corre tareas automáticas cada vez que alguien abre un PR.
CLI	Programa que se usa escribiendo comandos en la terminal, sin ventanas ni botones.
Node.js	El programa que permite ejecutar JavaScript fuera del navegador.
TypeScript	JavaScript con "etiquetas" de tipo (esto es un número, esto un texto). Ayuda a detectar errores antes de ejecutar.
Linter (ESLint)	Un corrector automático de estilo y errores comunes, como el corrector ortográfico de Word.
Test	Un mini programa que comprueba que otro funciona. Ej.: "si compro 2 de $10, el total es $20".
LLM / modelo	Una inteligencia artificial de texto, como ChatGPT, Gemini o Claude.
API	Una "ventanilla" por internet donde un programa le pide algo a otro (ej.: le pedimos a Gemini que revise).
API key	La contraseña para usar esa ventanilla. Nunca debe quedar escrita en el código.

Token	Un pedacito de texto (≈ 4 letras). Los modelos cobran y miden por tokens.
JSON	Un formato de texto ordenado para datos: `{"severidad": "alto", "linea": 42}`.
Determinístico	Que siempre da el mismo resultado con la misma entrada. Una calculadora lo es; una IA no del todo.
Prompt	El texto con instrucciones que se le manda a la IA.

3. Qué hace Guardian: los comandos

Todo se usa desde la terminal, dentro de la carpeta de cualquier proyecto que use git.

Lo más simple: `guardian` solo, o en tus palabras

```
$ guardian
¿Qué querés hacer?
  1. Revisar mis cambios                ← Enter
  2. Revisar y corregir automáticamente
  3. Preguntar sobre el código
  4. Elegir o cambiar la IA
  5. Ver la ayuda completa

$ guardian "revisá mis cambios con la IA local"
→ guardian review --use ollama
🤖 Usando: IA local · Ollama · qwen2.5-coder:14b (privada: el código no sale de tu máquina)
```

La primera vez aparece un menú para elegir la IA (replay, local chica o grande, o un proveedor: Gemini, Claude, Grok, GPT u otra). Si la elegida no está configurada, explica cómo hacerlo. La elección se guarda y la próxima vez arranca directo. Siempre se avisa con qué IA se está trabajando.

> **¿Cómo entiende las frases?**
>
> Con un "enrutador" por palabras clave, **sin IA**: "revisá" → review, "corregí" → review con corrección, "aprendé que…" → aprender, "con la IA local" → `--use ollama`. Si no reconoce la intención, lo toma como una pregunta. Es instantáneo, gratis y predecible, y muestra el comando equivalente para que la persona lo aprenda.

Corrección automática (con tu permiso)

1. Después de revisar, Guardian pregunta si querés que corrija.
2. Guarda una **foto** de cada archivo, y la IA corrige solo lo marcado.
3. **Valida** volviendo a revisar: problemas antes → después y si pasan los tests.
4. Muestra el diff y elegís: **commit + push**, **deshacer** (todo vuelve exactamente a como estaba) o **dejarlo sin commitear**. Por defecto, deshacer.

> **La red de seguridad**
>
> Si después de corregir sigue BLOQUEADO, la opción de subir aparece como "NO recomendado" y pide una segunda confirmación. Esto salió de una prueba real: el modelo local "arregló" una contraseña del frontend con `process.env`, que en el navegador no existe, y la validación lo detectó.

`guardian review` — revisar los cambios

Mira lo que cambiaste, busca problemas y da un veredicto. Es el comando principal.

```
$ guardian review

Chequeos determinísticos
  ✓ Secretos              1 ms
  ✓ ESLint              1.5 s
  ✓ Tipos (TypeScript)  1.5 s
  ✓ Tests               0.8 s

Hallazgos (1)
  CRÍTICO  src/routes/products.ts:40  [asincronia]  G-4e781f
    inventory.reserve es asíncrona y se llama sin await: la respuesta sale
    antes de reservar el stock y se puede vender algo que no hay.
    → await inventory.reserve(items);

Decisión:  BLOQUEADO
  Regla "critico-bloquea": 1 hallazgo crítico (la regla exige 1 o más)
```

Opciones útiles: `--no-llm` (sin IA, solo chequeos fijos) · `--json` (salida para máquinas) · `--base main` (comparar contra otra rama) · `--pre-push` (solo commits; lo usa el hook).

`guardian ask "pregunta"` — preguntarle al proyecto

Responde citando el archivo y la línea exacta. Sirve para alguien nuevo en el equipo.

```
$ guardian ask "¿Dónde se validan los datos que llegan a la API?"

Se validan en src/validation.ts con esquemas de zod y la función parse
(src/validation.ts:26-33). Las rutas la usan en src/routes/carts.ts:41, 55, 73 y 95.

Archivos consultados: src/validation.ts, src/routes/carts.ts, ...
Tiempo: 13.2 s · 11824 tokens de entrada / 286 de salida
```

`guardian chat` — charlar con el código

Una conversación en la terminal: preguntás, repreguntás (“¿y dónde se usa eso?”) y responde citando archivo y línea. Para salir: `/salir`. `/fuentes` muestra qué archivos leyó.

```
vos › ¿Cómo se calcula el costo de envío?
guardian › Con calculateShipping en src/services/pricing.ts:31 ...
vos › ¿y dónde se usa esa función?
guardian › En src/services/pricing.ts:45 (dentro de calculateTotals) y en tests/pricing.test.ts ...
```

El truco para que no invente

Antes de preguntarle a la IA, Guardian busca **sin IA** en qué líneas aparece cada nombre de función y se lo pasa como dato. Así el modelo cita ubicaciones reales. Sin esto, el modelo local de 14B inventaba lugares donde “se usaba” la función.

`guardian aprender "regla"` — enseñarle al equipo

Guarda una regla del equipo que se le pasa a **cualquier** IA en cada revisión. Con `--patron` la regla es **fija**: se detecta siempre, sin IA.

```
$ guardian aprender "Los precios nunca se calculan en el frontend" --archivos "public/**"
$ guardian aprender "Sin scripts inline en el HTML" --patron "<script>" --archivos
"public/**/*.html"
$ guardian aprender --lista
```

`guardian dismiss G-xxxxxx "motivo"` — "esto no es un error"

Si Guardian marca algo que el equipo considera correcto (un falso positivo), se descarta con un motivo. Queda guardado y Guardian no lo vuelve a reportar.

```
$ guardian dismiss G-757612 "Los tests de mayorista llegan en el PR siguiente (TICKET-123)"
✓ G-757612 descartado. Guardado en .guardian/memory.json
```

`guardian ia` — ¿qué inteligencia artificial estoy usando?

Muestra las tres opciones (replay, Gemini y Ollama), cuál se usa por defecto y si cada una está lista.

```
$ guardian ia
  ✓ replay   ← activa por defecto
      3 grabaciones de la demo. Sin key ni internet.
  ✓ gemini
      Key configurada · modelo gemini-3.6-flash. En la nube, la más precisa.
  ✓ ollama
      Listo · modelo qwen2.5-coder:7b (instalados: qwen2.5-coder:14b, qwen2.5-coder:7b).
```

`guardian --help` — la ayuda para principiantes

Escribir `guardian`, `guardian --help` o `guardian ayuda` muestra una guía completa en castellano: primeros pasos, cada comando con un ejemplo, cómo configurar cada IA, cómo leer el resultado y qué hacer si algo falla. `guardian review --help` muestra el detalle y ejemplos de un comando.

`guardian install-hook` / `uninstall-hook` — revisión automática antes de subir

Instala un "portero" en git: cada vez que hacés `git push`, Guardian revisa primero. Si está BLOQUEADO, el push se cancela.

Elegir la IA en cada comando: `--use`

Escribís	Usa	Qué necesita
`guardian review`	**replay** (por defecto)	Nada. Solo sirve para los ejemplos de la demo.

`guardian review --use ia` o `--use gemini`	**Gemini**, en la nube	Una API key gratis. Si falta, Guardian explica dónde sacarla y dónde pegarla.
`guardian review --use local` o `--use ollama`	**Ollama**, en tu compu	Ollama instalado y el modelo descargado. Si falta, Guardian da los comandos exactos.
`... --model qwen2.5-coder:14b`	Otro modelo de esa IA	Que esté descargado (`ollama pull ...`).

Agregar cualquier otra IA (ChatGPT, Claude, DeepSeek...)

Sin programar: en el archivo `.env` elegís un nombre y escribís 3 líneas. Apenas las guardás, Guardian la reconoce sola en `--use` y en `guardian ia`. La explicación completa, con la dirección de cada proveedor, está en `guardian ayuda otra-ia`.

```
IA_CHATGPT_URL=https://api.openai.com/v1      # dirección de la API
IA_CHATGPT_KEY=tu-clave                       # la clave de API
IA_CHATGPT_MODEL=nombre-del-modelo            # el modelo

$ guardian review --use chatgpt
```

Suscripción ≠ API

Pagar ChatGPT Plus o Claude Pro no alcanza: eso es para el chat en la web. Hace falta una **API key** de la consola para desarrolladores, que se paga por uso (una revisión cuesta centavos).

Cómo funciona por dentro

Casi todos los proveedores (OpenAI, DeepSeek, Groq, Mistral, OpenRouter, LM Studio) usan el mismo "idioma", llamado formato compatible con OpenAI, así que con **un solo adaptador** se cubren todos. Claude usa su propio formato: se agrega `IA_CLAUDE_FORMATO=anthropic` y Guardian usa el SDK oficial de Anthropic.

Orden de prioridad

`--use` en el comando > `LLM_PROVIDER` en el archivo `.env` > replay. Si pedís una IA con `--use` y no está lista, Guardian **corta antes de revisar** y te dice qué hacer, en vez de revisar a medias.

Ejemplo de un día normal

Ana arregla el checkout. Antes de subir, escribe `guardian review`. Guardian le avisa que olvidó un `await`. Lo corrige, vuelve a revisar, da APROBADO y hace push. Su compañero recibe el PR ya limpio y la revisión humana se concentra en lo importante.

4. Casos de uso

Quién	Cuándo	Qué hace Guardian
Programador	Mientras trabaja	`guardian review` en la terminal del editor. Feedback en segundos, antes de molestar a un compañero.
Programador	Al hacer `git push`	El hook revisa los commits. Si hay algo crítico, no deja subir.
Equipo	Al abrir un Pull Request	La GitHub Action revisa y deja un comentario en el PR con los hallazgos. Si bloquea, el check queda en rojo.
Persona nueva	Primera semana	`guardian ask` para entender dónde está cada cosa sin interrumpir a nadie.
Líder técnico	Al definir estándares	Escribe las reglas en `docs/CONVENTIONS.md` y en las "skills". Guardian las aplica igual para todos.
Equipo	Ante un falso positivo	`guardian dismiss`. El conocimiento queda guardado en el repo para todos.
Empresa con código sensible	Siempre	Usa Ollama: la IA corre en la propia computadora y el código no sale de ahí.

Qué tipo de problemas encuentra

Los chequeos fijos (sin IA)

- Contraseñas o API keys escritas en el código.
- Errores de estilo (ESLint).
- Errores de tipos (TypeScript).
- Tests que fallan.

La IA (con criterio)

- Un descuento aplicado dos veces.
- Precios calculados con decimales en vez de centavos.
- Un `await` olvidado.
- Riesgos de seguridad como XSS.
- Lógica nueva sin tests.

Por qué los dos tipos

Los chequeos fijos son rápidos, gratis y nunca se equivocan en lo que miden, pero solo ven lo que se les programó. La IA entiende la intención del código y encuentra errores de lógica, pero cuesta tokens, tarda más y puede equivocarse. Guardian usa lo fijo siempre y la IA solo cuando aporta.

5. El viaje de un cambio (el "pipeline")

Un **pipeline** es una cadena de pasos: la salida de uno es la entrada del siguiente, como una línea de montaje. Esto es lo que pasa cuando escribís `guardian review`:

1. **Recolector** — "¿Qué cambió?"
   Le pregunta a git qué líneas se agregaron o borraron y en qué archivos. Archivo: `git.ts`

▼

2. **Chequeos determinísticos** — las máquinas de medir
   Busca secretos, aplica las **reglas fijas del equipo** y corre lint, tipos y tests, todos a la vez. Después la IA revisa igual, para mostrar **todos** los problemas juntos (opcional: saltearla si ya hay un crítico, para ahorrar). Archivos: `checks/secrets.ts`, `checks/rules.ts`, `checks/commands.ts`

▼

3. **Constructor de contexto** — preparar la carpeta para el experto
   Junta el diff, los archivos tocados, los que estos usan, las convenciones, las skills y la memoria. **Tapa las contraseñas** antes de mandar nada. Archivos: `context.ts`, `skills.ts`, `memory.ts`

▼

4. **Agente revisor (IA)** — el catador
   Le manda todo al modelo y recibe una lista de hallazgos en JSON. Si la respuesta viene mal armada, reintenta una vez; si vuelve a fallar, sigue sin IA y avisa. Archivos: `reviewer.ts`, `llm/*`

▼

5. **Gate de decisión** — el jefe con reglamento
   Cuenta hallazgos por severidad y aplica reglas fijas: 1 crítico → BLOQUEADO; 3 altos → BLOQUEADO; 1 alto o medio → ADVERTENCIA. Dice qué regla aplicó. Archivo: `gate.ts`

Ejemplo completo con el escenario 04

1. Git dice: cambiaron `src/routes/products.ts` y `src/validation.ts`. **2.** No hay secretos; ESLint, tipos y tests pasan (por eso el bug es "sutil"). **3.** Se arma el contexto y se cargan las skills *backend* y *security* (no la de precios, porque no se tocaron precios). **4.** La IA responde: "`inventory.reserve` se llama sin `await`, severidad crítico". **5.** El gate aplica "1 crítico → BLOQUEADO". Total: unos 20 segundos con Gemini; 2 segundos en modo replay.

> **Para decir en la entrevista**
>
> "La IA opina, pero no decide. El gate es determinístico: con los mismos hallazgos siempre da la misma decisión, y siempre dice qué regla aplicó. Eso lo hace auditable y configurable por el equipo."

6. Cómo está organizado el proyecto

El repositorio tiene dos “paquetes” principales y algunas carpetas de apoyo. A esta forma de tener varios paquetes en un mismo repo se le llama **monorepo**.

```
Agente-Reviewer/
├── agent/              ← GUARDIAN (el revisor). Lo más importante.
│   ├── src/            ← el código
│   └── tests/          ← 129 tests que comprueban que funciona
├── storefront-demo/    ← La TIENDA de ejemplo (lo que Guardian revisa)
│   ├── src/            ← el servidor (API)
│   ├── public/         ← la página web (HTML, CSS, JS)
│   ├── tests/          ← 24 tests de precios, descuentos y stock
│   ├── docs/           ← convenciones y arquitectura de la tienda
│   └── .guardian/      ← configuración y skills de Guardian para esta tienda
├── demo/
│   ├── scenarios/      ← 4 cambios con errores a propósito (.patch)
│   ├── recordings/     ← respuestas reales de Gemini grabadas
│   └── run-demo.mjs    ← el script de “npm run demo”
├── docs/               ← decisiones, tutorial
├── .github/workflows/  ← la GitHub Action
├── .env.example        ← plantilla de configuración (qué IA usar)
└── README.md           ← cómo instalarlo y usarlo
```

Por qué hay una tienda adentro

El reto pedía un código de ejemplo de e-commerce para demostrar el sistema. La tienda es el “paciente”: Guardian es el “médico”. Los 4 escenarios de demo son enfermedades inventadas que se le aplican a la tienda para ver si el médico las diagnostica.

7. La tienda de ejemplo (storefront-demo)

Es una tienda chica pero real: catálogo de productos, carrito lateral, cupones de descuento y checkout. Se abre con `npm run store` en `http://localhost:3000`. Probá el cupón `BIENVENIDA10`.

Las piezas

Pieza	Qué hace	Comparación
`data/products.json`	Lista de productos con precio y stock.	El depósito.
`src/services/`	Las reglas del negocio: precios, descuentos, stock, checkout.	La cocina: donde se hace el trabajo.
`src/routes/`	La API: recibe pedidos de la página y responde.	Los mozos: toman el pedido y lo llevan a la cocina.
`src/validation.ts`	Revisa que los datos que llegan sean correctos.	El de seguridad en la puerta.
`public/`	La página web que ve el cliente.	El salón del restaurante.
`docs/CONVENTIONS.md`	Las reglas de la "empresa".	El manual interno.

La regla de oro: precios en centavos

En la tienda, $18,50 se guarda como `1850` (centavos). ¿Por qué? Porque las computadoras se equivocan con los decimales:

```
// En JavaScript (y en casi todos los lenguajes):
0.1 + 0.2   // da 0.30000000000000004  ← ¡no da 0.3!
10 + 20     // da 30, siempre exacto (por eso se usan centavos)
```

Con plata, ese error mínimo, repetido miles de veces, termina en diferencias reales. Por eso la convención dice "siempre centavos enteros", y el escenario 02 rompe esta regla a propósito.

Un pedacito de código: el descuento

```
export function calculateDiscount(subtotalCents, coupon) {
  if (!coupon) return 0;                              // sin cupón, no hay descuento
  if (coupon.type === "percent")
    discount = Math.round(subtotalCents * coupon.percent / 100); // 10% redondeado al centavo
  return Math.min(discount, subtotalCents);           // nunca más que el subtotal
}
```

Ejemplo

Carrito de $85,80 (8580 centavos) con `BIENVENIDA10`: 10% de 8580 = 858 centavos. Total: 8580 − 858 = 7722 centavos = $77,22.

Tests de la tienda

24 tests comprueban cosas como “10% de 999 centavos redondea a 100”, “el descuento nunca supera el subtotal” y “dos compras seguidas no pueden vender más stock del que hay”. Son tan buenos que atrapaban los bugs “fáciles”, y por eso los escenarios de demo ponen los errores en partes *sin* tests (ver capítulo 9).

8. El agente, archivo por archivo

Todo el código de Guardian está en `agent/src/`. Está escrito en TypeScript. Vamos en el mismo orden del pipeline. Cada ficha tiene: **qué hace**, **comparación**, **código** (simplificado para que se entienda) y **qué tenés que saber**.

`cli.ts` — la puerta de entrada

Qué hace Lee lo que escribiste en la terminal (`review`, `ask`...) y llama a la parte del programa que corresponde. Usa la librería commander.

Comparación La recepción de un hotel: escucha qué necesitás y te manda al lugar correcto.

```
program.command("review")
  .option("--json", "salida en JSON")
  .action(async (opciones) => {
    const reporte = await runReview({ cwd: process.cwd() });  // hace la revisión
    console.log(renderReport(reporte));                       // la muestra linda
    process.exitCode = reporte.decision === "BLOQUEADO" ? 1 : 0; // 1 = "frená"
  });
```

Saber El exit code es un número que todo programa devuelve al terminar: 0 significa "todo bien", otro número significa "algo pasó". El hook y la Action miran ese número para saber si cortar.

`review.ts` — el director de orquesta

Qué hace Ejecuta los 5 pasos del pipeline en orden y arma el reporte final. Mide cuánto tarda cada paso.

Comparación El director: no toca ningún instrumento, pero dice quién entra y cuándo.

```
const cambios  = collectChanges(...);            // 1. ¿qué cambió?
const fijos    = await chequeosFijos(...);       // 2. secretos, lint, tipos, tests
if (hayCritico && config.ahorrar) avisar("Se omitió la IA"); // opcional
else {
  const contexto = buildContext(...);             // 3. preparar lo que ve la IA
  const ia = await reviewWithLLM(...);           // 4. preguntarle a la IA
}
const decision = decide(hallazgos, reglas);       // 5. el gate decide
```

Saber Si la IA se intentó y falló (Google caído, sin cuota), un APROBADO se baja a ADVERTENCIA con la regla `ia-no-disponible`. No traba al equipo, pero tampoco aprueba a ciegas.

`git.ts` y `diff.ts` — el recolector

Qué hace `git.ts` le pregunta a git qué cambió. Decide contra qué comparar: si hay cambios sin guardar, contra el último commit; si no, los commits de tu rama contra `main`. `diff.ts` lee ese diff y anota qué líneas son **nuevas** y en qué número de línea quedaron.

Comparación Un "buscar diferencias" entre dos fotos.

```
// Así se ve un diff. diff.ts se queda con las líneas que empiezan con "+":
@@ -45,3 +45,4 @@
   const lines = buildLines(items, catalog);
+  inventory.reserve(items);                ← línea nueva, número 40
   res.status(201).json(...);
```

Saber Solo se analizan las líneas **agregadas**. Así no se culpa al programador por código viejo que no tocó.

`checks/secrets.ts` — el detector de contraseñas

Qué hace Busca en las líneas nuevas cosas con forma de contraseña o API key, usando **expresiones regulares** (patrones de texto).

Comparación El detector de metales del aeropuerto: busca formas conocidas.

```
// Algunos patrones (simplificados):
{ nombre: "API key de Google",      patron: /AIza[0-9A-Za-z_-]{35}/ }
{ nombre: "Credencial hardcodeada", patron: /(api_key|password|token)... = "..."/ }

// Ejemplo que SÍ detecta:   const API_KEY = "prod_9f8a7b6c5d4e3f2a1b0c";
// Ejemplo que NO detecta (está bien):  const apiKey = process.env.API_KEY;
```

Saber El reporte muestra solo los primeros 4 caracteres del secreto (`prod… (25 caracteres)`) para no volver a filtrarlo. Esta misma lista de patrones se usa para **tapar** secretos antes de mandar código a la IA.

`checks/commands.ts` — lint, tipos y tests

Qué hace Mira el `package.json` del proyecto para saber qué comandos existen (`npm run lint` , `npm test` …) y los corre **en paralelo**. Si alguno falla, crea un hallazgo con las últimas líneas del error.

Comparación Tres inspectores trabajando al mismo tiempo en vez de uno detrás del otro.

Saber Severidades fijas: tipos o tests rotos = **crítico** (bloquea); errores de lint = **alto**. Correr en paralelo hace que el total tarde lo que el más lento (~1,5 s), no la suma.

`context.ts` — el armador del contexto

Qué hace Arma el "paquete" que va a leer la IA, en orden de importancia y con un tope de tamaño: convenciones → diff → archivos modificados (con números de línea) → archivos que estos importan.

Comparación Preparar la carpeta para un consultor externo: le das lo necesario, ordenado, y sin papeles confidenciales.

```
// Lo que NUNCA se manda, aunque nadie lo pida:
const SIEMPRE_IGNORADO = [".env", ".env.*", "node_modules/", "dist/", "package-lock.json"];
// + lo que diga el archivo .guardianignore del proyecto
// + todo secreto se reemplaza por [SECRETO REDACTADO]
```

Saber El tope (~80.000 caracteres ≈ 20.000 tokens) controla costo y tiempo. Lo que no entra se informa como "no enviado".

`skills.ts` — los manuales especializados

Qué hace Una skill es un archivo de texto (`.guardian/skills/precios.md`) con criterios de revisión para un área. Guardian carga solo las que corresponden a los archivos que cambiaron.

Comparación Un médico clínico que consulta el manual de cardiología solo si el problema es del corazón.

```
// En .guardian/config.json:
"skills": {
  "ecommerce-pricing": ["src/services/pricing.ts", "src/services/discounts.ts"],
  "frontend":          ["public/**"]
}
// Si cambiaste public/app.js → se carga "frontend", NO "ecommerce-pricing".
```

Saber Dato real: con skills, el modelo más barato de Gemini encontró el bug del escenario 04 como crítico en 1,5 s. Sin skills había bajado la severidad.

`memory.ts` — la memoria del equipo

Qué hace Guarda en `.guardian/memory.json` los hallazgos que el equipo descartó con `guardian dismiss`. Después: (1) los oculta y no cuentan para la decisión, y (2) se los pasa a la IA como "excepciones conocidas".

Comparación Las notas que deja el turno anterior: "esto ya lo revisamos, está bien así porque…".

Saber Es lo que hace que Guardian mejore con el uso sin reentrenar la IA. El archivo se guarda en el repo, así lo comparte todo el equipo. No se pueden descartar tests o tipos rotos: eso se arregla, no se discute.

`reviewer.ts` — el agente revisor

Qué hace Escribe el prompt, se lo manda al modelo y valida la respuesta con la librería zod: que sea JSON, que cada hallazgo tenga archivo, línea, severidad válida, explicación y sugerencia.

Comparación Un formulario con campos obligatorios: si el consultor lo entrega incompleto, se lo devolvés una vez para que lo corrija.

```
// Lo que la IA debe devolver:
{ "findings": [ { "file": "src/routes/products.ts", "line": 40,
                  "severity": "critico", "category": "asincronia",
                  "explanation": "...", "suggestion": "await inventory.reserve(items);" } ] }

// Si no cumple → reintento con el error explicado.
// Si falla dos veces → se sigue sin IA y se avisa (nunca se cae el programa).
```

Saber Si la IA cita un archivo que no cambió, ese hallazgo se descarta (control de "alucinaciones"). El ID del hallazgo (`G-4e781f`) lo genera Guardian, no la IA.

`llm/` — los enchufes para cada IA

Qué hace Define una interfaz común: "dame un texto y devolveme la respuesta". Hay tres implementaciones:

Archivo	Proveedor	Cuándo usarlo
`gemini.ts`	Google Gemini (nube, capa gratis)	Rápido de configurar con una key gratuita.
`ollama.ts`	Ollama (IA en tu compu)	Código confidencial: no sale de tu máquina.
`replay.ts`	Respuestas grabadas	Probar la demo sin key ni internet.

Comparación Un adaptador universal de enchufes: el cargador (el revisor) es el mismo, cambia solo el enchufe.

Saber Se elige con `--use` en el comando, o por defecto con `LLM_PROVIDER` en el `.env` ; si no hay nada, replay. Antes de revisar, Guardian verifica que la IA esté lista (por ejemplo, que Ollama esté abierto y tenga el modelo); los textos de ayuda están en `llm/help.ts` . Gemini se llama directo por internet, sin librería extra, y la key va en un "header", nunca en la dirección web. Ante saturación (error 503) reintenta; ante cuota agotada (429) no, porque cada intento gasta cuota.

`gate.ts` — el jefe con reglamento

Qué hace Cuenta los hallazgos por severidad y aplica las reglas en orden. Primero las que bloquean, después las que advierten. La primera que se cumple decide.

```
for (const regla of reglasOrdenadas) {
  if (cantidad[regla.severidad] >= regla.minimo)
    return { decision: regla.decision, regla: regla.nombre };   // ej: "critico-bloquea"
}
return { decision: "APROBADO" };                                // ninguna regla se cumplió
```

Saber — Las reglas se cambian en `.guardian/config.json` sin tocar código. Ejemplo: un equipo más exigente puede decir "2 medios ya bloquean".

`ask.ts` — preguntas sobre el código

Qué hace — 1) Saca las palabras clave de la pregunta. 2) Puntúa cada archivo según cuántas veces aparecen. 3) Manda los más relevantes a la IA (hasta ~10.000 tokens). 4) Devuelve la respuesta con citas `archivo:línea`.

Ejemplo — "¿Dónde se validan los datos?" → palabras clave: *validan, datos* → el archivo con más puntos es `validation.ts` → se le manda a la IA.

Saber — Es una búsqueda simple por palabras, no "semántica". Respeta las mismas reglas de privacidad que `review`.

El resto, en una línea cada uno

`report.ts` — Dibuja el reporte con colores en la terminal.

`config.ts` — Lee `.guardian/config.json`, lo valida con zod y completa lo que falte con valores por defecto.

`hook.ts` — Instala y desinstala el "portero" del `git push`.

`types.ts` — Define la forma de los datos (qué es un "hallazgo", un "reporte") y genera los IDs.

`env.ts` — Lee el archivo `.env` (qué IA usar y su key).

`timer.ts` — Mide cuánto tarda cada paso.

`guide.ts` — El texto de `guardian --help`, de `guardian ayuda otra-ia` y los ejemplos de cada comando.

`actions.ts` — El flujo de cada comando: elegir la IA, revisar, ofrecer corregir y decidir (push, deshacer o dejar).

`fixer.ts` — Corrección automática: pedirle a la IA el archivo corregido, aplicar, deshacer (rollback) y commit + push.

`menu.ts` — Los menús numerados y el "🤖 Usando: ...". Escribe la clave en el `.env` si la pegás en el menú.

`prefs.ts` — Guarda la IA elegida en tu carpeta de usuario (no en el repo).

`nl.ts` — Traduce frases ("revisá mis cambios con la IA local") al comando equivalente.

`project.ts` — En un repo con varios proyectos, elige cuál se revisa (la carpeta con `.guardian/` o `docs/CONVENTIONS.md`), para usar SUS reglas.

`checks/rules.ts`	Las reglas fijas del equipo: patrones que se buscan en las líneas nuevas, sin IA.
`chat.ts`	El chat en la terminal (lee línea por línea, comandos /salir, /fuentes, /nuevo).
`llm/custom.ts`	Lee del `.env` las IAs que agregó el usuario (`IA_<NOMBRE>_...`) y les habla en formato OpenAI o Anthropic.

9. Los 4 escenarios de demo

Cada escenario es un **patch**: un archivo que describe un cambio, como si lo hubiera hecho un programador. La demo lo aplica sobre una copia de la tienda y corre Guardian.

Escenario	Qué cambia	Quién lo detecta	Resultado
01-clean	Filtro por categoría, bien hecho, con validación y tests.	Nadie (no hay error).	APROBADO
02-pricing-float	Descuento mayorista calculado con decimales y el cupón aplicado dos veces.	La IA (los tests pasan porque no prueban 10+ unidades).	BLOQUEADO 2 críticos
03-security	Tarjetas armadas con `innerHTML` (riesgo XSS) y una API key escrita en el código.	El detector de secretos (sin IA).	BLOQUEADO
04-subtle	Nueva ruta "comprar ahora" que reserva stock sin `await`.	La IA (compila, pasa lint y tests).	BLOQUEADO 1 crítico

Un descubrimiento que vale oro para la entrevista

Al preparar los escenarios pasó algo revelador: los tests de la tienda **atrapaban** los bugs "obvios", y entonces la IA ni se llamaba. Por eso los errores de 02 y 04 se pusieron en código **sin tests**. Ese es justamente el caso real donde un revisor con IA aporta valor: lo que se escapa de los tests.

> **Y otro: la IA encontró un error en el escenario "limpio"**
>
> Al regrabar las respuestas, la IA marcó un problema en el 01, que se suponía correcto: el filtro pasaba la categoría a minúsculas, pero el catálogo comparaba contra el texto original. Una categoría "Cocina" nunca iba a coincidir. Era un bug real y se corrigió.

¿Qué es el XSS del escenario 03?

`innerHTML` mete texto en la página **como si fuera código**. Si alguien logra que el nombre de un producto sea `<img src=x onerror="robarDatos()">`, la página ejecuta ese código en el navegador de cada cliente. La forma segura es `textContent`, que lo trata como texto común.

> **Una decisión que cambió probando**
>
> Al principio, si el secreto ya bloqueaba, Guardian no llamaba a la IA (para ahorrar), y la XSS del 03 aparecía recién en la segunda revisión. Al probarlo con cambios reales decidiste que es mejor ver **todos los errores juntos**: ahora la IA revisa siempre (el secreto se le manda tapado) y el ahorro quedó como opción (`ia.saltearSiHayCriticos`). Además, la regla fija `sin-innerHTML` marca la XSS aun sin IA.

10. Demo, replay, hook y GitHub Action

`npm run demo -- all`

El script `demo/run-demo.mjs` hace, para cada escenario: copia la tienda a la carpeta `.demo-work/` (que git ignora, así el repo queda limpio), aplica el patch, corre Guardian y al final muestra un resumen "esperado vs. obtenido".

```
✓ 01-clean           APROBADO                 APROBADO     0   1.8 s
✓ 02-pricing-float   BLOQUEADO                BLOQUEADO    3   1.8 s
✓ 03-security        BLOQUEADO                BLOQUEADO    1   1.8 s  sin IA
✓ 04-subtle          ADVERTENCIA o BLOQUEADO  BLOQUEADO    1   1.8 s
4/4 escenarios con la decisión esperada.
```

El sandbox: `npm run demo -- sandbox`

Crea una copia limpia de la tienda en `.demo-work/sandbox`. Ahí podés romper lo que quieras y correr `guardian review` sin tocar el proyecto. Los casos para probar están en `docs/CASOS-DE-PRUEBA.md`. La demo también acepta `--use`: `npm run demo -- all --use local`.

Modo replay y `--record`

Con `--record`, Guardian guarda en `demo/recordings/` las respuestas reales de Gemini. Con `LLM_PROVIDER=replay`, las reproduce sin internet ni key. Así cualquiera puede probar la demo en 3 minutos.

> **Comparación**
>
> Es como grabar un partido: la jugada fue real, pero la podés ver otra vez sin que los jugadores estén en la cancha. Si el código cambió desde la grabación, Guardian lo avisa.

El hook pre-push

`guardian install-hook` crea un pequeño script en `.git/hooks/pre-push`. Git lo ejecuta solo antes de cada push:

- Revisa **solo los commits que se van a subir**, no lo que tengas a medio escribir.
- Si el resultado es BLOQUEADO, cancela el push. El servidor no recibe nada.
- Si Guardian no está instalado o falla, avisa y deja subir. Una herramienta de ayuda no debería trabar al equipo.
- Para saltearlo una vez: `git push --no-verify`.

Se probó con un push real: el commit con un secreto fue rechazado y el remoto quedó intacto.

La GitHub Action

El archivo `.github/workflows/guardian.yml` le dice a GitHub: "cada vez que se abra un Pull Request, instalá el proyecto, corré `guardian review --json` y comentá el resultado en el PR". Si es BLOQUEADO, el check queda en rojo. La key de Gemini se guarda como **secret** del repo, nunca en el código.

> **Detalle de seguridad que se corrigió**
>
> La primera versión pegaba el reporte dentro del código del script. Un hallazgo con comillas especiales podía romperlo o inyectar código. Ahora el reporte se lee desde un archivo. La parte que comenta se probó localmente; en GitHub falta probarla con un PR real.

11. Por qué este stack y no otro

"Stack" es el conjunto de tecnologías elegidas. El reto daba libertad total, así que la elección es parte de lo que evalúan.

Elección	Por qué
Herramienta de terminal (CLI)	Revisar código exige leer el repo local, correr los tests del proyecto y poder frenar un `git push`. Todo eso pasa en la compu del programador. Un CLI lo hace directo y el mismo programa sirve para la terminal, el hook y la CI.
Node.js + TypeScript	Es el mismo lenguaje que la tienda. Un solo lenguaje para todo, y los tipos ayudan a detectar errores.
Pocas librerías	commander (comandos), zod (validar datos), picocolors (colores), ignore (filtros tipo .gitignore), vitest (tests). Menos dependencias = más fácil de instalar y de entender.
Gemini sin SDK	Se llama a la API directamente: se ve exactamente qué se manda y no se suma una librería más.
Varios proveedores	Gemini para empezar gratis, Ollama para privacidad, replay para probar sin nada. Cambiar de uno a otro es cambiar una línea del `.env`.

Alternativas que se descartaron (te las van a preguntar)

Alternativa	Por qué no como núcleo	Dónde sí sumaría
n8n	Necesita un servidor, no puede frenar un push local, el código viaja a otro lugar y la lógica en "cajitas" es difícil de testear.	Después de la decisión: avisar por Slack o crear un ticket con el JSON del reporte.
Google ADK, LangGraph (frameworks de agentes)	Hoy hay una sola llamada a la IA dentro de pasos fijos. Un framework suma complejidad sin aportar.	Si el revisor pasa a usar herramientas en un loop (capítulo 14).
Power Apps / Power Automate	Son para apps de negocio de Microsoft. No acceden al repo ni a git y requieren licencias.	No encaja en herramientas para programadores.
Que la IA decida	No sería auditable ni repetible: la misma pregunta puede dar otra respuesta.	La IA opina; el gate decide.
Comprar una herramienta (CodeRabbit, Copilot review)	El reto pedía construir. Lo propio suma convenciones del negocio, skills, memoria en el repo, gate auditable y opción 100% local.	En una empresa real, conviene evaluarlas antes.

> **Para decir en la entrevista**
>
> “Elegí un CLI porque revisar código pasa en la máquina del desarrollador: hay que leer el repo, correr sus tests y poder frenar un push. n8n lo usaría después, para notificar con el resultado; no para revisar.”

12. Costo, tiempo, privacidad y operabilidad

El reto pedía explicar cómo se balancean estas 4 condiciones, que a veces chocan entre sí.

Condición	Qué se hizo	Dato concreto
Costo	Capa gratuita de Gemini, Ollama gratis, replay gratis. Opción de no llamar a la IA si ya está bloqueado. Skills solo si aplican. Tope de tamaño del contexto.	$0. Una revisión usa ~6.000 a 12.000 tokens de entrada.
Tiempo	Chequeos en paralelo. El modelo se eligió midiendo calidad y velocidad.	Chequeos ~1,5 s. Con IA, 10 a 60 s según demanda. Replay ~2 s.
Privacidad	Secretos tapados antes de enviar. Nunca se envía `.env`, `node_modules`, binarios. `.guardianignore`. Ollama para código sensible.	La capa gratis de Gemini puede usar los datos para mejorar sus productos: para código real, Ollama o plan pago.
Operabilidad	Instalación con `npm install` + `npm link`. Replay sin key. README y tutorial paso a paso.	Demo completa en ~3 minutos.

Cómo se eligió el modelo (medido, no supuesto)

Modelo	Tiempo	Calidad
gemini-3.8-flash (el más nuevo)	60–80 s o saturado	La mejor
gemini-3.6-flash ← elegido	~11–20 s	Muy buena
gemini-3.5-flash-lite	~2 s	Se equivocó sin skills; con skills, acertó
Ollama qwen2.5-coder:14b (local, RTX 5080)	~9–16 s	Buena: 3 de 4 escenarios; no ve el cupón doble y da falsos positivos
Ollama qwen2.5-coder:7b (local)	~2–10 s	Floja: 2 de 4; solo encuentra el `await` faltante

Cuando las condiciones chocan

- **Costo vs. feedback completo:** primero se priorizó el costo (sin IA si ya había un crítico); probándolo se priorizó ver todos los errores juntos, y el ahorro quedó configurable.
- **Velocidad vs. calidad:** se eligió el modelo intermedio, no el más rápido ni el mejor.
- **Privacidad vs. comodidad:** Gemini es lo más cómodo; Ollama es lo más privado. El sistema permite ambos.

- **Seguridad vs. no frenar al equipo:** si la IA falla, no se aprueba (ADVERTENCIA), pero tampoco se bloquea.

13. Qué hiciste vos y qué hizo la IA

El reto lo pregunta explícitamente y dice que no evalúan si usaste IA, sino **el criterio con que la usaste y qué tan bien entendés lo que construiste**. La respuesta honesta es una fortaleza.

Tuyo La idea, el diseño y las decisiones

- **La especificación completa** con la que arrancó todo: el nombre, los 5 comandos, el pipeline de 5 pasos, el gate determinístico, las skills, la memoria con `dismiss`, los 3 proveedores (incluido replay con `--record`), los 4 escenarios con sus bugs, la GitHub Action, la estructura de carpetas y el contenido del README.
- **La elección de qué atacar**: revisar + decidir como núcleo, preguntas como secundaria, y preferir "algo enfocado y bien hecho".
- **Las 4 condiciones** como eje de diseño.
- **La forma de trabajar**: plan por fases, aprobación antes de cada fase, commit por fase, código simple y comentado en español.
- **Decisiones durante el trabajo**: usar Gemini como primer proveedor, primero mantener la regla "crítico determinístico → sin IA" y después cambiarla para ver todos los errores juntos, descubrir probando que el modelo chico daba un falso positivo y que en el monorepo no se cargaban las reglas de la tienda, pedir el chat y el aprendizaje, pedir que se corrijan los errores antes de entregar.

IA (Claude) La implementación y el detalle técnico

- **Escribir todo el código** y los tests, fase por fase, siguiendo tu especificación.
- **Decisiones de implementación** dentro de tu diseño: Express 5, cómo detectar secretos con regex, el ID de cada hallazgo, correr chequeos en paralelo, cómo armar el contexto, la redacción de secretos.
- **Hallazgos al probar**: medir modelos y elegir gemini-3.6-flash; descubrir que los tests atrapaban los bugs y mover los escenarios a código sin tests; bajar a ADVERTENCIA cuando la IA falla; no reintentar ante cuota agotada; que el hook revise solo commits.
- **Errores propios que después se corrigieron**: ejemplos inválidos en el README, una GitHub Action insegura, `ask` devolviendo JSON y el hook sin probar.
- **Documentación**: README, tutorial y esta guía.

Cómo decirlo en la entrevista

"La idea y el diseño son míos: definí qué atacar, la arquitectura del pipeline, que el gate fuera determinístico, las skills, la memoria y los escenarios. La implementación la hice con Claude, trabajando por fases: yo aprobaba cada plan y revisaba cada entrega. Varias decisiones salieron de probar, como elegir el modelo midiendo o mover los bugs a código sin tests. Y hubo errores de la IA que detectamos y corregimos antes de entregar."

Importante

Si en tu especificación inicial también usaste ayuda de una IA, decilo igual. Lo que valoran es que entiendas cada parte, y para eso está esta guía.

14. Límites y qué harías con más tiempo

Límites conocidos (mejor decirlos vos antes que te los encuentren)

- La capa gratuita de Gemini permite unas **20 revisiones por día** por modelo.
- **Los modelos locales son menos precisos:** con Ollama, el 7B solo encontró el bug del escenario 04 y el 14B encontró además los decimales del 02 (no el cupón doble). Ambos dan falsos positivos en cambios correctos. Ver `docs/CASOS-DE-PRUEBA.md`.
- El revisor **propone** correcciones pero **no las prueba** (por eso la capacidad 2 del reto es parcial).
- `ask` busca por palabras clave, no entiende sinónimos ni recuerda preguntas anteriores.
- El contexto tiene un tope: cambios muy grandes pueden quedar incompletos.
- En el modo hook, lint y tests corren sobre los archivos actuales, no exactamente sobre el commit.
- La GitHub Action no se probó todavía con un PR real.

La gran mejora: un revisor más "agente"

Una crítica honesta que pueden hacerte: **hoy es un pipeline con un paso de IA, más que un agente autónomo**. La IA recibe un paquete armado y responde una vez. Un agente de verdad **investiga** cuando le falta información.

> **Cómo sería**
>
> 1. **Herramientas para la IA**: leer otro archivo, buscar dónde se usa una función, correr un test puntual, escribir un test nuevo para **probar** que el bug existe.
> 2. **Un loop con límites**: máximo 6 pasos, 40.000 tokens o 60 segundos. Cada paso queda registrado para ver cómo llegó a la conclusión.
> 3. **Modo "auto"**: revisión rápida por defecto; investigación solo si el cambio toca precios o checkout, o si encontró algo grave que conviene confirmar.
> 4. **Hallazgos verificados**: "este test que escribí falla" vale más que "me parece que hay un error". Eso además cumpliría la capacidad 2 del reto.
> 5. **Medirlo**: `npm run eval` comparando el modo rápido y el agéntico en bugs encontrados, falsos positivos, tokens y tiempo.

Otras ideas

- Exponer Guardian como **servidor MCP** para que los agentes del editor (Claude Code, Cursor) lo usen solos.
- Usar **gitleaks / Semgrep** para detectar más tipos de secretos y patrones peligrosos sin IA.
- Conectar con **n8n o Slack** para avisar al equipo y armar métricas.
- Búsqueda semántica para `ask`.

15. Preparación para la entrevista

Preguntas probables y respuestas cortas

1. ¿Qué parte del ciclo atacaste y por qué?

> "La revisión de código, que es lenta e inconsistente y deja pasar bugs. Elegí revisar y decidir como núcleo porque se refuerzan entre sí: el feedback sirve al desarrollador y la decisión sirve al equipo. Sumé preguntas sobre el código en versión simple. Preferí dos capacidades bien hechas."

2. ¿Por qué no decide el LLM?

> "Porque no es repetible ni auditable. La IA encuentra y explica; un gate con reglas fijas decide y dice qué regla aplicó. Si el equipo quiere ser más estricto, cambia una regla en el config."

3. ¿Qué pasa con un falso positivo?

> "El desarrollador corre `guardian dismiss` con un motivo. Queda en `memory.json`, versionado en el repo: no se vuelve a mostrar y se le pasa a la IA como excepción. El sistema mejora con el uso sin reentrenar nada. Tests o tipos rotos no se pueden descartar."

4. ¿Cómo protegés el código de la empresa?

> "Tres capas. Antes de enviar, se tapan los secretos y nunca se mandan .env ni dependencias. El equipo puede excluir carpetas con .guardianignore. Y para código sensible está Ollama: la IA corre local y nada sale de la máquina. La capa gratis de Gemini es para probar."

5. Un modelo marcó un error y otro no. ¿Cuál tiene razón?

> "Hay que mirar el hallazgo, no el modelo. Me pasó: el modelo chico marcó 'un JSON con información sensible' y el grande no. Era la memoria de Guardian, sin nada sensible: era un falso positivo del chico. En vez de 'enseñárselo' a todos, lo arreglé de raíz: Guardian ya no le manda a la IA sus archivos internos ni binarios."

6. ¿Qué es "razonable" en tiempo de respuesta?

> "Para el pre-push, menos de un minuto, porque es una pausa natural del desarrollador. En CI, un par de minutos. Los chequeos fijos tardan ~1,5 s y la IA entre 10 y 60 s según la demanda."

7. ¿Es un agente o un script con IA?

> "Hoy es un pipeline con un agente revisor en un paso. Lo diseñé así por costo y previsibilidad. La evolución natural es darle herramientas para investigar en un loop acotado, con un modo automático que solo se active en cambios riesgosos."

¿Cómo elige el usuario la IA? ¿Y si no la tiene instalada?

> "Con `--use` en cada comando: replay por defecto, `--use ia` para Gemini y `--use local` para Ollama. Si la elegida no está lista, Guardian corta antes de revisar y explica qué falta: dónde sacar la key o cómo instalar Ollama y bajar el modelo. Y `guardian --help` explica todo para alguien que nunca lo usó."

¿Probaste la IA local? ¿Qué tal anda?

> "Sí, con Ollama en una RTX 5080. El 14B encontró 3 de 4 escenarios en 9 a 16 segundos; el 7B, 2 de 4. Gemini fue el más preciso. Es el trade-off: privacidad total con algo menos de precisión. Probarlo además me hizo mejorar cosas: los modelos chicos repetían hallazgos y perdían la pregunta en textos largos."

¿El agente puede corregir solo? ¿No es peligroso?

> "Corrige solo con permiso y siempre con red: guarda una foto de cada archivo, corrige, vuelve a revisar para validar y me muestra el diff. Elijo subir, deshacer o dejarlo. Si sigue bloqueado, recomienda deshacer y pide confirmar dos veces. Eso cubre la capacidad del reto de detectar, corregir y validar. Y nunca sube nada sin que yo lo elija."

¿Por qué el lenguaje natural no usa IA?

> "Porque para entender 'revisá mis cambios' alcanzan palabras clave: es instantáneo, gratis, predecible y lo puedo testear. Usar un modelo ahí sería más lento y caro, y con replay no funcionaría. Uso la IA donde aporta criterio, no para traducir comandos."

¿El agente aprende? ¿Reentrenás la IA?

> "No reentreno nada. Aprende guardando el conocimiento del equipo en el repo y pasándoselo a la IA en cada revisión: reglas en palabras, descartes de falsos positivos y reglas fijas. Medí que los modelos locales no siempre aplican una regla escrita en palabras, pero sí la misma como regla fija. Por eso lo innegociable va determinístico y la IA queda para el criterio. Y como vive en el repo, si cambiamos de modelo, lo aprendido sigue."

¿Por qué la IA revisa aunque ya haya un error crítico?

> "Primero lo hacía al revés, para ahorrar. Probándolo vi que el desarrollador recibía un error por vez, y es peor experiencia. Ahora reporta todo junto: el secreto se le manda tapado, así que es seguro. El ahorro quedó como opción en la configuración."

¿Y si la empresa ya paga ChatGPT o Claude?

> "Lo agregan en el `.env` con tres líneas y lo usan con `--use`, sin tocar código. Un único adaptador cubre a todos los proveedores compatibles con OpenAI; Claude va con el SDK oficial. Y la ayuda aclara algo que confunde mucho: la suscripción del chat no incluye la API."

8. ¿Qué harías distinto?

> "Validaría las correcciones con tests generados, mediría con un set de evaluación y probaría la Action en un PR real desde el principio."

Guion para la demo (30 minutos)

Minutos	Qué mostrar
0–3	El problema y la idea en una frase (capítulo 1). La comparación de la fábrica.
3–6	La tienda funcionando: `npm run store`, cupón, checkout.
6–14	`npm run demo -- all`. Detenerse en 02 (la IA ve lo que los tests no ven) y en 04 (el `await`).
14–17	`guardian dismiss` y volver a revisar: la memoria en acción.
17–19	`guardian ask` con una pregunta en vivo.
19–21	El hook: un push que se cancela.
21–26	Arquitectura (capítulo 5), decisiones y las 4 condiciones.
26–30	Límites, mejora agéntica y qué hiciste vos / la IA.

Plan B

Si Gemini está saturado en vivo, corré lo mismo con `--use local` o `--use replay`. Decilo con naturalidad: "esto es exactamente para lo que existe el modo replay".

16. Machete de comandos

Comando	Para qué
`npm install`	Instala todo y compila Guardian.
`npm link`	Deja disponible el comando `guardian` en cualquier carpeta.
`cp .env.example .env`	Crea la configuración (por defecto, modo replay).
`npm run store`	Abre la tienda en http://localhost:3000.
`npm test`	Corre los 153 tests (129 del agente + 24 de la tienda).
`npm run demo -- all`	Corre los 4 escenarios con resumen.
`npm run demo -- 02-pricing-float`	Corre un solo escenario.
`npm run demo -- all --record`	Regraba las respuestas con el modelo real.
`guardian review`	Revisa tus cambios.
`guardian review --no-llm`	Solo chequeos fijos, sin IA.
`guardian review --json`	Resultado en formato para máquinas.
`guardian ask "pregunta"`	Pregunta sobre el código (necesita Gemini u Ollama).
`guardian dismiss G-xxxxxx "motivo"`	Descarta un falso positivo.
`guardian install-hook`	Activa la revisión antes de cada push.
`guardian uninstall-hook`	La desactiva.
`guardian --help`	Guía completa para principiantes.
`guardian ia`	Qué IA está activa y cuáles están listas.
`guardian review --use ia`	Revisar con Gemini.
`guardian review --use local`	Revisar con Ollama (agregá `--model qwen2.5-coder:14b` para el grande).
`guardian`	Menú principal: revisar, corregir, preguntar, elegir IA.
`guardian "revisá mis cambios"`	Lo mismo, en tus palabras.
`guardian review --corregir`	Revisar y corregir automáticamente (con tu confirmación).
`guardian chat --use local`	Charlar con el código (repreguntas, /fuentes, /salir).

`guardian aprender "regla"`	Enseñarle una regla; con `--patron` es fija.
`guardian ayuda otra-ia`	Cómo agregar ChatGPT, Claude u otra IA.
`guardian review --use <tu-ia>`	Revisar con una IA que agregaste en el `.env`.
`npm run demo -- sandbox`	Copia limpia de la tienda para probar a mano.
`git push --no-verify`	Sube salteando el hook una vez.

El archivo `.env`

```
LLM_PROVIDER=replay          # por defecto; se cambia en cada comando con --use
GEMINI_API_KEY=              # gratis en aistudio.google.com/apikey
GEMINI_MODEL=                # vacío = gemini-3.6-flash
OLLAMA_MODEL=qwen2.5-coder:7b
LLM_TIMEOUT_MS=60000
```

Las 3 ideas para llevarte

1. **Lo fijo primero, la IA después:** lo barato y seguro siempre corre; la IA solo cuando aporta.
2. **La IA opina, las reglas deciden:** decisiones explicables y configurables.
3. **Mejora con el uso:** skills y memoria versionadas en el repo, sin reentrenar nada.

Guía actualizada con el estado del repositorio al 23/10/2026 (incluye menú, lenguaje natural, corrección automática, --use, IAs propias, chat, aprender, monorepo y las pruebas con Ollama). Si cambiás el código, revisá que los ejemplos sigan coincidiendo.